Logistic regression

→ Why logistic regression? Is it regression?

→ too many topic where dependent variables are limited?

→ Assume two classes: 0 and 1

→ Spam / not spam (email)

→ Cancer / non-cancer

→ Customer behaviour prediction: Sad / happy

• Coded as binary dependent variable (two class problem)

0 → negative class

1 → positive class

→ Can we use linear regression for classification?

→ It can be formulated by this

$h(x_i) = \hat{y}_i >= 0.5$, predict positive class or 1

$\hat{y}_i < 0.5$, predict 0

→ For present problem:

1. The output $h(x_i)$ is unbounded.

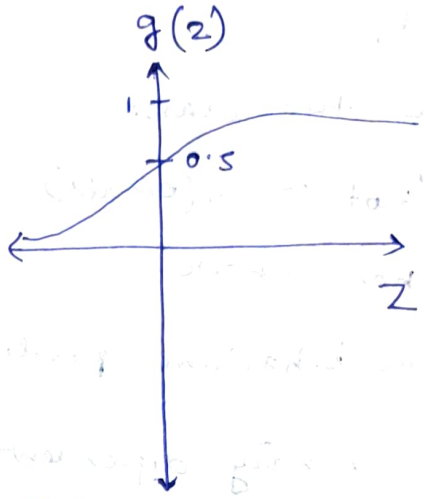
$h(x_i) > 1$ or < 0

2. Then fixing threshold at 0.5 is reasonable or not!

Solution: Search a function can bound the output between 0 to 1.

→ Logistic function is a smooth S-shaped curve that maps any real number into the range (0,1).

$$g(z) = \frac{1}{1+e^{-z}}$$



$g(0) = 0.5$ (value at 0)

Derivative:

$$g'(z) = g(z)(1 - g(z))$$

$g(z) \to 0$ if $z \to -\infty$

$g(z) \to 1$ if $z \to \infty$

Interpretation of hypothesis

$$0 \le \hat{y}_i \le 1$$

$h(x_i)$ = estimated probability that $y_i = 1$ for a given x_i

$$\hat{y}_i = h(x_i) = 0.7$$

It means

There is 70% chance that x_i belongs to $y_i = 1$;
30% chance that x_i belongs to $y_i = 0$

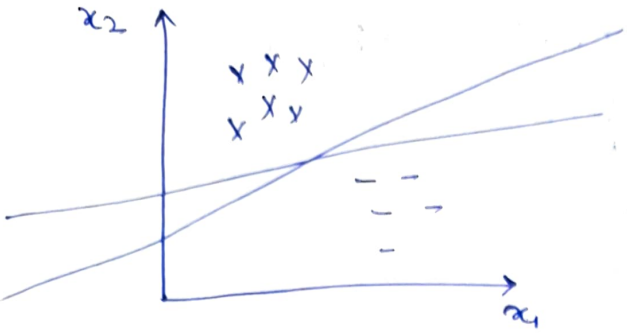
$$h_w(x_i) = P(y=1 \mid x_i, w)$$

probability that $y = 1$ for a given w

$$P(y=1 \mid x_i; w) + P(y=0 \mid x_i; w) = 1$$

$$P(y=0 \mid x_i; w) = 1 - P(y=1 \mid x_i; w)$$

Classification (binary)



→ We have to find decision boundary

→ Here, linear decision boundary



Non-linear decision boundary

Simple case (two feature)

DB: line (higher dimensional hyperplane)

$w_1 x_1 + w_2 x_2 + w_0 \geq 0$ (one side of line or on the line)

$w_1 x_1 + w_2 x_2 + w_0 < 0$ (another side of the line)

$X_i = [x_{i,1} \quad x_{i,2}]$

$y_i = 0/1$ (positive / negative class)

$X_i \in 0$ class | $w_1 x_{i,1} + w_2 x_{i,2} + w_0 < 0$; $w_j x_{i,j} < 0$

$X_i \in 1$ class | $w_1 x_{i,1} + w_2 x_{i,2} + w_0 \geq 0$; $w_j x_{i,j} \geq 0$

$x_{i,0} = 1$

So, we have to find a decision boundary (DB) so that all training samples placed at the proper side of the DB

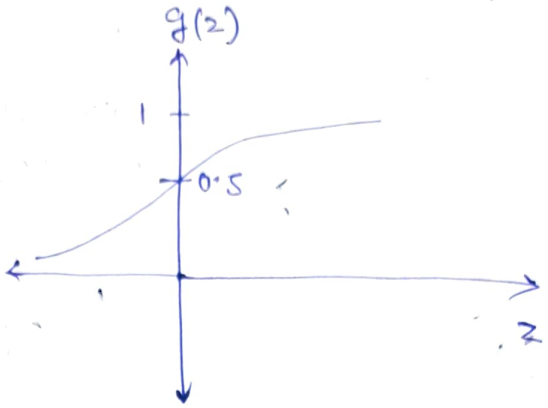
Back to logistic regression

$x_i = [x_{i,0}\ x_{i,1}\ x_{i,2}\ \cdots\ x_{i,n}]$ $\quad x_{i,0} = 1$

$y_i = 0 / 1$

$\hat{y}_i = g(w^T x_i)$

$$w = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{bmatrix}$$



Suppose, we have to predict $y_i = 1$

then $\hat{y}_i \geq 0.5$

then $g(w^T x_i) \geq 0.5$

$w^T x_i \geq 0$

Suppose, we have to predict $y_i = 0$

$\hat{y}_i < 0.5$

$g(w^T x_i) < 0.5$

$w^T x_i < 0$

Dataset

x_1	x_2	y(in label)	class label	y
2	0.6	spam	1	0
6	3	no spam	2	1
1	2	spam	1	0

positive class 1

negative class 0

logistic regression (cost function)

$$J(W) = \frac{1}{2m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2 \quad \text{— MRSS/MSE}$$

$$\hat{y}_i = g(W^T x_i) \qquad g(W^T x_i) = \frac{1}{1 + e^{-W^T x_i}}$$ } non-linearity involves

Here, $$J(W) = \frac{1}{2m} \sum_{i=1}^{m} (y_i - g(W^T x_i))^2$$

→ non-convex cost function

→ If we proceed with MSE/MRSS, we are getting non-convex ones (having multiple local minima)

→ solution (log-loss function / cross entropy loss)

weight updation rule with MSE cost function

SGD

$$W_j^{itr+1} = W_j^{itr} - \frac{\partial}{\partial W_j} J(W)$$

$$= W_j^{itr} + \alpha (y_i - \hat{y}_i) x_{i,j} \hat{y}_i (1 - \hat{y}_i)$$

Another cost function for logistic regression

$$J(W) = \frac{1}{m} \sum_{i=1}^{m} \text{cost}(\hat{y}_i, y_i)$$

$$\text{cost}(\hat{y}_i, y_i) = \begin{cases} -\log(\hat{y}_i) & \text{if } y_i = 1 \\ -\log(1 - \hat{y}_i) & \text{if } y_i = 0 \end{cases}$$

- if sample from positive class ($y_i = 1$), the loss is $-\log(\hat{y}_i)$ → High penalty if predicted probability of positive class is small.

- If true label is 0, the loss is $-\log(1-\hat{y}_i)$

- High penalty if predicted probability of 0 ~~negative~~ class is small.

$$J(w) = \frac{1}{m}\sum_{i=1}^{m} \text{cost}(\hat{y}_i, y_i)$$

$$= \frac{1}{m}\sum_{i=1}^{m} -y_i \log(\hat{y}_i) - (1-y_i)\log(1-\hat{y}_i)$$

$$= -\frac{1}{m}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

The name is log-loss function.

Weight updation using log-loss cost function for logistic regression

---

SGD:

$$w_j^{itr+1} = w^{itr} - \alpha(\hat{y}_i - y_i)x_{i,j}$$

Algorithm representation of batch GD for logistic regression

Step1: Randomly initialize $w_0, w_1, \ldots w_n$

Step2: Calculate $\hat{y}_i$ for all training samples

Step3: Calculate $J(w)$ for itr^{th} epoch

Step4: Update all parameters $w_0, w_1, \ldots w_n$

Step5: Repeat steps 2 to 5 until
$|J^{itr}(w) - J^{itr-1}(w)| > p$ or
number of epochs $< n_0$.

Step6 Apply threshold to take final decision

Step7 Report the value of $J^{thr}(H)$ and classification accuracy.

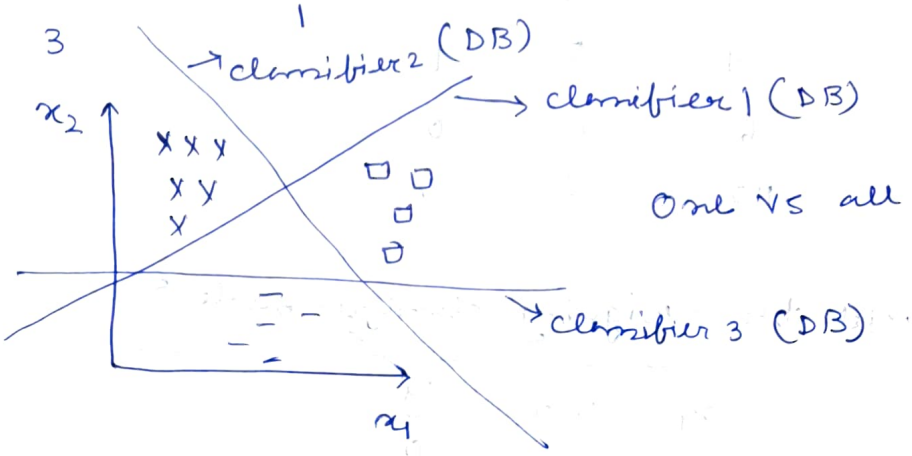
Some to do list same as linear regression

- Split train/test/validation
- Normalized the data
- Hyperparameter tunning
- Choose best model for final training.

Multi-class logistic regression

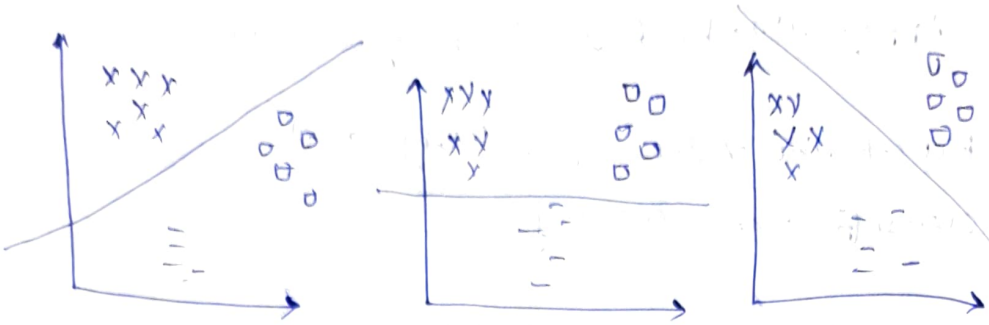
x_1	x_2	class label
2	3	1
4	5	2
6	7	3
1.2	3	1

Three or more classes.



One Vs all

Design three different logistic regression model for each class.



Classifier model 1

Classifier model 2

Classifier model 3

Dataset for designing classifier model 1

x_1	x_2	y
2	3	1
4	5	0
6	7	0
1.2	3	1

Dataset for classifier model 2 (design)

x_1	x_2	y
2	3	0
4	5	1
6	7	0
1.2	3	0

Dataset for designing classifier model 3

x_1	x_2	y
2	3	0
4	5	0
6	7	1
1.2	3	0

m = number of training samples

$X_1, X_2, \cdots X_m$

n = number of features

$X_j = [x_{j,1} \quad x_{j,2} \quad \cdots \quad x_{j,n}]$

C = number of classes

$K \in \{1, 2, \cdots C\}$

One vs all

→ Train a logistic regression model $h^K(x_i)$ for each class K to predict probability that $y_i = K$ for a given x_i

→ Testing phase to know class label for unknown pattern

class label

$x_{test} \rightarrow ?$

$h^1(x_{test})$

$h^2(x_{test})$

⋮

$h^K(x_{test})$

⋮

$h^C(x_{test})$

Choose K for $\max_K h^K(x_{test})$

→ parameters are already estimated for all hypothesis in training phase.

K-fold cross validation

- Shuffle the dataset randomly
- Split the dataset in K groups / fold (approximately equal)
- For each group
  - → Take out as a hold out or test dataset
  - → Take the remaining groups as training dataset
  - → Fit a model on the training set and evaluate it on test set.
  - → Retain the evaluation score and discard the model.
- Report the performance score for each group and average all the performance score.
  - Some portion of training set used as validation set
  - K usually 5 to 11
  - all samples uses one time to train and test.

Confusion matrix

C x C matrix, C is number of classes.

predicted \ Actual	(+)	(−)
(+)	TP 560	FP 60
(−)	FN 50	TN 330

(1) Accuracy = $\frac{TP+TN}{TP+FN+TN+FN}$

(2) precision = $\frac{TP}{TP+FP}$

Out of all positive predicted samples how many are truly positive.

(3) Recall = $\frac{TP}{TP+FN}$

How many of actual positive cases we are able to predict correctly with our model.